Monash University

FIT5202 – Data Processing for Big Data (2026 S2)

Assignment 2B: Using Streaming and ML for Smart Transport & Predictive Parking Visualisation

Due: 23:55 Monday 02/November/2026 (Week 14 Monday) | Weight: 15% of the final marks

---

Background (The same as A2A)

Urban congestion and parking search inefficiencies represent major challenges for modern smart cities. Studies show that up to 30% of downtown traffic congestion is caused by drivers cruising to find an open parking bay. Traditional smart parking systems suffer from the "stale parking" dilemma: displaying available parking spaces at the moment of search is of limited utility to a motorist still minutes away in traffic, as those vacant bays will likely be occupied before the driver arrives.

To address this challenge, metropolitan transport authorities are transitioning towards integrated, predictive smart navigation systems. By coupling historical road traffic volume, corridor transit performance, and vehicle fleet characteristics with IoT parking sensor data, transport authorities can anticipate future parking pressure. Accurate forecasting allows municipal dispatchers to implement dynamic traffic calming, advise drivers on arrival-time availability, and optimise urban mobility.

Big data architectures and distributed machine learning let us analyse vast, multi-source transport datasets; handle spatiotemporal correlations; isolate meaningful predictive features from sensor noise; and build models capable of scalable inference.

Objective of the Project

In this assignment, you will act as a lead data scientist for a municipal smart transport agency (such as Transport Victoria or the City of Melbourne). In Assignment 1, you explored large-scale distributed batch processing and spatiotemporal joins on historical mobility records. Then, in Assignment 2A, we built regression models using Apache Spark MLlib to predict destination parking availability (available_spots) based on corridor traffic flow, vehicle composition, temporal congestion windows, and IoT parking sensor data.

In Assignment 2B, you will deploy your trained and persisted ML pipeline model from Assignment 2A into an end-to-end real-time data streaming pipeline using Apache Kafka and Apache Spark Structured Streaming. You will simulate live incoming sensor feeds (traffic telemetry and parking updates), join them with static municipal registries, perform dynamic real-time parking availability inference, compute windowed streaming aggregations, persist outputs to Parquet streams, forward processed predictions back to Kafka, and construct an interactive operational dashboard for municipal traffic and parking dispatchers.

Key Information

Submission links can be found on Moodle.

> **IMPORTANT:** A2B has a **compulsory** interview/demo component to be conducted in Weeks 14–15. The teaching team marks A2B submissions **strictly during** your demo session. **Failure to attend this demo will result in 0 marks** (for A2B).

(The demo sign-up sheet will be announced in Week 12 or 13. Please pay attention to the unit announcements in the final teaching week. If you have an approved special consideration or extension, demo sessions will be arranged according to the approved extension; however, this may delay your final result release.)

The Datasets

The datasets can be downloaded from Moodle as a compressed archive:

- **streaming_traffic_data.csv:** Real-time traffic sensor readings capturing incoming vehicle counts, classifications, speed brackets, and directional headings across key corridors.
- **streaming_parking_bays.csv:** Parking bay infrastructure and baseline capacity information stream.
- **area.json and street.json:** Mapping tables for parking areas and streets to parking sensor devices.
- **sites.csv:** Geographic coordinates (latitude and longitude) of traffic monitoring stations.
- **vehicle_types.xlsx:** Vehicle profile categories and classifications in Excel format.
- **Metadata:** The metadata schema is identical to Assignment 2A, since you need to use your trained model. Please refer to the A2A appendix for the metadata.

What You Need to Achieve

Component	Description	Approach
A2B Use Case	Ingest streaming traffic sensor data via Kafka, join with static municipal spatial metadata, apply the trained MLlib regression model to predict real-time parking availability (available_spots), compute windowed streaming aggregations, and build a live interactive dashboard for the municipal operation centre.	Apache Kafka + Spark Structured Streaming + Real-Time Visualisation

Architecture

The following figure represents the overall architecture of the assignment setup across Part A and Part B. Part A consists of data preparation, feature engineering, and model training/persisting. Part B consists of Kafka streaming ingestion, Spark Structured Streaming inference/aggregation, Parquet persistence, and Kafka consumer dashboard visualisation.

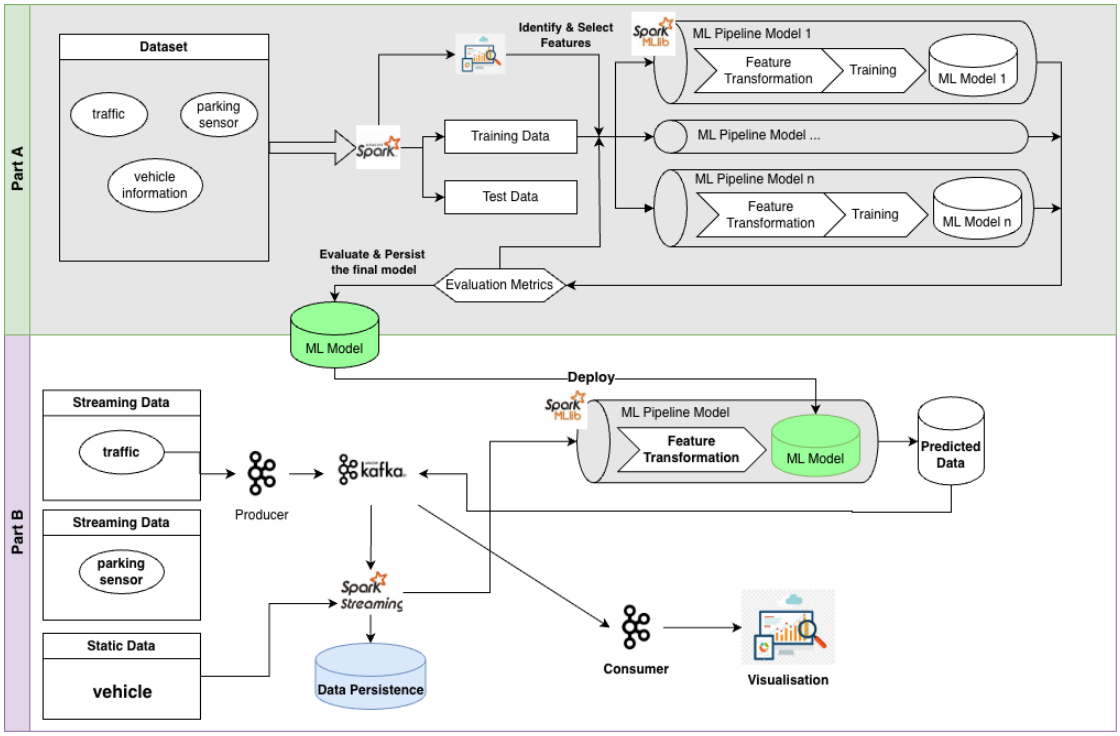


Fig. 1: Overall Architecture for Assignment 2 (This assignment is Part B)

In both parts, you must implement the solutions using PySpark DataFrame and MLlib for data processing and machine learning pipelines. Pandas is strictly permitted only for local plotting and data visualisation. Please follow the steps to document the processes and write the code in your Jupyter Notebook.

Getting Started

- Download the datasets from Moodle.
- Download the template notebooks for submission purposes:
  - A2B-Task1_producer.ipynb: for streaming data production using Kafka Producer.
  - A2B-Task2_spark_streaming.ipynb: for consuming, processing, predicting, and aggregating data using Spark Structured Streaming.
  - A2B-Task3_consumer.ipynb: for consuming predictions using Kafka and visualising.
- For submission, append your authcate ID at the end of the filename (e.g., A2B-Task1_producer_xxxx0000.ipynb, where xxxx0000 is your authcate ID).
- You will use Python 3+ and PySpark 4.1.0+ for this assignment (the environment is provided as a Docker image, identical to labs).

A2 Part B Specification

Your task in this part is to build a real-time smart transport intelligence dashboard for the municipal traffic and parking operations centre to oversee incoming vehicle flow and predicted destination parking availability so that they can proactively implement traffic calming and parking guidance.

Part 1: Producing the Data (10%)

In this task, we will implement an Apache Kafka producer in Python to simulate real-time data streaming. Spark and parallel data processing should **not** be used in this section, as we are simulating roadside edge sensors that lack distributed processing capabilities. However, you can use the standard Python library to preprocess the data.

1. Every 5 seconds, load a batch of records from streaming_traffic_data.csv and streaming_parking_bays.csv. You should keep a pointer in the files reading process and advance it per read. The data must be read in chronological order.

Notes:

- If you performed some strategies to reduce the amount of data by aggregation or filtering, you can do it here using Python standard library or pandas (no Spark).
- A batch refers to a minimum set of data that your model requires to make a prediction, for example, one time_bin for a site.)

2. Add the current timestamp (traffic_ts, parking_ts) to the records in the batch to represent the real-time event transmission time.

3. Send your batch of traffic sensor data to two Kafka topics with appropriate names (e.g., traffic_stream, parking_stream).

Deliverable: Save your code in Assignment-2B-Task1_producer.ipynb.

Part 2: Streaming Application using Spark Structured Streaming (40%)

In this task, you will implement Spark Structured Streaming to consume the telemetry from Task 1, perform real-time prediction using your trained ML pipeline model from Assignment 2A, execute tumbling window aggregations, and persist the results.

> **Important:**
> • This task uses PySpark Structured Streaming with PySpark DataFrame APIs and PySpark MLlib.
> • You will load your saved pipeline model from Assignment 2A (from Part 3 tuning, e.g., models/best_tuned_parking_model) to make predictions and persist the results. If you didn't complete the model tuning, it's also fine to load the model from part 2.
> • Event Time Note: In an operational deployment, traffic_ts represents event time. Because streaming is simulated and your model was trained with temporal features, you may derive temporal attributes (e.g., hour, day of week, peak period) from traffic_ts or the original time column.

Tasks:

1. Write code to create a SparkSession that: 1) uses four CPU cores with a proper application name; 2) sets the session timezone to Australia/Melbourne; 3) ensures a checkpoint location has been set.

2. Write code to define the data schema for the data files. Load the datasets (area.json, street.json, Sites.csv, and vehicle_types.xlsx etc.) into DataFrames. (You can reuse your code from 2A.) In

urban transport management, traffic telemetry is dynamic and streaming, whereas sensor monitoring locations, road network topology, and parking zone capacities are static registries.

3. Using the Kafka topic from the producer in Task 1, deserialise and ingest the streaming data into Spark Streaming, assuming all data comes in string format except for the traffic_ts and parking_ts columns as numeric/timestamp types. Convert the streaming DataFrame attributes to their appropriate types.

4. Use a watermark on the timestamps. If data points are received 30 seconds late, discard the data. (Note: In a local test environment, late arrivals may rarely happen, but the watermark must be properly specified).

5. Perform the necessary transformations you used in Assignment 2A (e.g., peak period categorisation, zone partitioning, traffic pressure ratios, encoding, vector assembly). If you built an end-to-end pipeline in Assignment 2A that encapsulates all transformation stages, you can pass the parsed stream directly into the pipeline model. (You can reuse your code in A2A for this task).

6. Load your pipeline model and perform predictions of available_spots. Then, perform the following streaming aggregations using tumbling windows:

- **a) Critical Parking Shortage Alert:** Make predictions and print monitoring sites or corridors where severe parking shortages are forecasted (predicted available_spots < 10 or occupancy > 90%) every 5 seconds.
- **b) Corridor Flow & Parking Availability:** Every 10 seconds, compute and print the total traffic volume and average predicted available spots grouped by directional heading (e.g., inbound towards CBD vs outbound).
- **c) Precinct Mobility Breakdown:** Every 30 seconds, for each parking precinct (based on your geospatial partitioning), print the total traffic volume, average predicted available spots, and the count of areas categorised as Low (0–5 spots), Medium (6–15 spots), and High (>15 spots) availability.

7. Save the aggregated and predicted data from Task 6 to Parquet files as streams with checkpointing enabled.

(Hint: Parquet files support streaming writing/reading. The directory should continuously update as new micro-batches arrive).

8. Read the Parquet files from Task 7 as data streams and publish them to Kafka output topics with appropriate names. (Note: You shall read the Parquet files as a streaming DataFrame and send messages to the Kafka topic when new data appears in the Parquet file).

Deliverable: Save your code in Assignment-2B-Task2_spark_streaming.ipynb.

Part 3: Consuming Data using Kafka and Visualise (30%)

In this task, we will implement an Apache Kafka consumer in Python to consume the processed data from Part 2. You shall not use (and don't need) Apache Spark in this section.

> **Important:**
> In this part, Kafka consumers are used to consume the streaming data published from Task 2.8.
> For students who can't complete task 2, **we still encourage you to complete this part**. You can simply aggregate available_spots from the streaming parking bay and use the sum as your prediction (instead of your model). Please inform your interviewer if you decide to go with this path.

1. Load static site data and consume streaming records from Part 2.

2. Create a dashboard with 3 plots to visualise the real-time streaming data from Part 2:

- **a) Plot 1 – Availability Trend (Line / Bar Chart):** A dynamic plot tracking predicted available parking spots and traffic volumes over time across key corridors.
- **b) Plot 2 – Availability Distribution (Histogram):** A live histogram showing the cumulative distribution of predicted available spots across monitored zones.
- **c) Plot 3 – Geospatial Bubble or Choropleth Map Plot (Melbourne Smart City View):** A bubble or choropleth map plot centred on Melbourne CBD and monitored precincts. Each bubble represents a sensor site/parking precinct, where bubble location is determined by coordinates/zone, bubble size represents traffic volume, and bubble colour represents predicted parking availability (e.g., Red = Low Availability / Critical Shortage, Yellow = Medium Availability, Green = High Availability). You may use any library (e.g., Plotly, Folium, Seaborn, Matplotlib).
  - For reference: BubbleMap (https://samplecode.link/bubble-map/) or Choropleth Map (https://samplecode.link/choropleth-map/)
  - You can search and use additional GeoJSON or OpenStreetMap (OSM) data to assist your plot.

(Note: Ideally, for an authentic dashboard user experience, the 3 diagrams need to be updated in real time simultaneously. You may need to do a bit of searching/research to make it work.)

You can refer to the following resources:

- Python Graph Gallery (https://samplecode.link/ or https://python-graph-gallery.com/)
- Matplotlib (https://matplotlib.org/)
- Plotly (https://plotly.com/python/)

Deliverable: Save your code in Assignment-2B-Task3_consumer.ipynb.

Part 4: Demo and Interview (20%)

> **IMPORTANT:** The interview is compulsory, and we only mark your A2B during the interview. No marks will be awarded if the interview is not attended (0 marks for the whole A2B, not just this section).

The demo/interview session details will be announced/arranged in the last teaching week. Please pay attention to the unit announcement email and Ed forum.

Each demo is roughly 10 minutes. You have 5–6 minutes to show your application; then, your marker will ask 3–4 questions to assess your understanding. Please arrive at your allocated demo session a few minutes early and ensure your laptop/application is working correctly. **You can start streaming,**

processing, and visualisation before joining the breakout room, and show your interviewer to speed up the demo.

Note: For students who couldn't complete the visualisation part, you can demonstrate the raw console output of your Spark streams, but a maximum score of 15 will apply.

The demo/interview is marked on a 5-level scale:

Performance Standard	Mark
The demo is working, and the student has a competent understanding	**20**
Working demo on console and/or partial understanding	**15**
The demo is not working and/or partial understanding	**10**
The demo is not working and/or low understanding	**5**
No attendance or can't answer most of the questions	**0**

The demo session will be arranged in Weeks 14 and 15 (Exam weeks 1 and 2 after the SWOT).

Submission A2B

You should submit your final version of the assignment solution via Moodle. You must submit a single ZIP file named based on your authcate ID (e.g., abcd1234.zip).

The zip file should contain:

- Assignment-2B-Task1_producer_authcate.ipynb
- Assignment-2B-Task2_spark_streaming_authcate.ipynb
- Assignment-2B-Task3_consumer_authcate.ipynb

Strict Format Requirement: The file in submission should be a ZIP file and not any other kind of compressed folder (e.g., .rar, .7zip, .tar). Please do not include the data files or your model in the ZIP file.

Assignment Marking Rubric

Detailed mark allocation is available in each task. For complex tasks and explanation questions, you will receive marks based on the quality of your work.

In your submission, the Jupyter Notebook file should contain the code and its output. It should follow programming standards, readability of the code, and organisation of code. Please refer to the PEP 8 — Style Guide for Python Code (https://peps.python.org/pep-0008/). Penalty applies if your code is hard to understand with insufficient comments.

Other Information

Where to Get Help

You can ask questions about the assignment in the Assignments section in the Ed Forum, which is accessible on the unit's Moodle Forum page. This is the preferred venue for assignment clarification-type questions. You should check this forum regularly, as the responses of the teaching staff are

"official" and can constitute amendments or additions to the assignment specification. Also, you can attend scheduled consultation sessions if the problem and the confusion are still unresolved.

Searching and learning on commercial websites/forums (e.g., Quora, Stack Overflow) is allowed. However, you should not post/ask assignment questions on those forums.

Plagiarism and Collusion

Plagiarism and collusion are serious academic offences at Monash University. Students must not share their work with other students. Students should consult the policy linked below for more information:
https://www.monash.edu/students/academic/policies/academic-integrity
See also the video linked on the Moodle page under the Assignment block.

Students involved in collusion or plagiarism will be subject to disciplinary penalties, which can include:

- The work not being assessed
- A zero grade for the unit
- Suspension from the University
- Exclusion from the University

Late Submissions and Special Consideration

ALL Special Consideration, including within the semester, is now handled centrally. This means that students MUST submit an online Special Consideration form via Monash Connect. For more details, please refer to the Unit Information section in Moodle.

There is a 5% penalty per day, including weekends, for a late submission. Also, the cut-off date is 7 days after the due date. No submission will be accepted (i.e., zero mark) after the cut-off date unless you have a special consideration.

Mark Release and Review

- Mark will be released within 10 business days after the submission deadline. For assessments with demo/interview, the "submission deadline" refers to the date when the majority of the interview is completed; not the due date in Moodle.
- Reviews and disputes regarding the mark will be accepted a maximum of 7 days after the release date (including weekends).

Generative AI Statement

As per the University's policy on the guidelines and practices pertaining to the usage of Generative AI:

AI & Generative AI tools may be used SELECTIVELY within this assessment. Where used, AI must be used responsibly, clearly documented and appropriately acknowledged (see Learn HQ).

Any work submitted for a mark must:

1. Represent a sincere demonstration of your human efforts, skills, and subject knowledge for which you will be accountable.

2. Adhere to the guidelines for AI use set for the assessment task.
3. Reflect the University's commitment to academic integrity and ethical behaviour.

Inappropriate AI use and/or AI use without acknowledgement will be considered a breach of academic integrity.

The teaching team encourages students to apply their own critical thinking and reasoning skills when working on the assessments with assistance from GenAI. Generative AI tools may produce inaccurate content, which could have a negative impact on students' comprehension of big data topics.

Data Source Acknowledgement

The dataset is a remix based on several real-world datasets and synthetic data. We thank the original contributors of the datasets.